In-Place Test-Time Training on Qwen3-4B: A Controlled Ablation of the Test-Time Update

Rayhaan Khan
Mumbai AI Lab
Mumbai, India

Abstract

In-Place Test-Time Training (In-Place TTT) turns the MLP down-projection of selected Transformer layers into a fast weight that is updated from the input during inference. Its published evaluation on Qwen3-4B reports RULER gains over a matched continued-pretraining control. That comparison cannot say whether the gain comes from the update at inference or from training with the module. We separate the two with an ablation planned before the runs. We continually pretrained Qwen3-4B-Base for 3.0B tokens at 32k context with and without the released recipe, on identical data in identical order, with two seeds. All checkpoints were evaluated on RULER from 4k to 64k with 500 instances per task. The ablation evaluates the TTT checkpoint a second time with its inner learning rate set to zero, so the fast weights never move. Averaged over seeds, disabling the update changes the RULER mean by at most 0.4 points at lengths up to 64k with plain rotary embeddings. At 64k with YaRN the update lowers the mean by 2.5 points. The gains of the TTT arm over the control, 1 to 4 points, are present with the update disabled. Training with the module explains them. The TTT arm also trains differently. Its gradient norm is 100 to 1,000 times the control's, concentrated in six small target convolutions, and its loss stays 0.007 nats above the control's on the same batches. The released chunk size of 4,096 is the worst of three tested. The clipping threshold reported for the method removes the update instead of bounding it. Code, configurations, per-instance predictions and the study plan are released.

1 Introduction

Transformers do not change their weights while they read. Test-time training relaxes this. A subset of parameters is updated from the input during inference, so the model can adapt to a long document as it goes (Sun et al., 2020; Hardt & Sun, 2023; Sun et al., 2024). In-Place Test-Time Training (Feng et al., 2026) applies the idea to pretrained language models with few changes. The MLP down-projection in one layer out of six becomes a fast weight. After a 4,096-token chunk, the model computes a self-supervised target from the chunk, applies one gradient step to that projection, and reads the next chunk with the updated weights. The authors continually pretrained Qwen3-4B with the module and compared it with a control trained on the same data. On RULER (Hsieh et al., 2024) the TTT model led by 0.6 points at 32k and by 4.4 points at 64k.

That comparison leaves the mechanism open. A TTT-trained model differs from its control in two ways. Its shared weights were trained together with a module that changes the inputs of the following layers, and at inference its fast weights adapt to the prompt. The published numbers combine both differences. The method is motivated by the second difference, adaptation at test time, and that difference has not been measured on its own.

The distinction decides how the method should be used. If the update is the source of the gain, longer inputs should help more and the inner learning rate at inference should matter. The update should also transfer beyond the training length. If training with the module

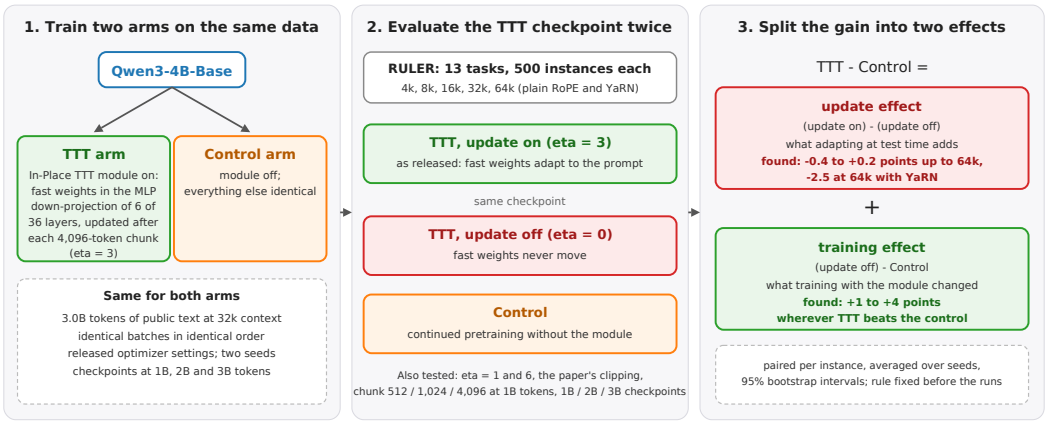


Figure 1: What was done and tested. Two arms are trained from Qwen3-4B-Base on identical data, with and without the In-Place TTT module (left). The TTT checkpoint is evaluated on RULER twice, with its test-time update on and off, next to the control (middle). The difference between TTT and the control then splits into an update effect and a training effect (right). The update effect is close to zero at all lengths up to 64k with plain RoPE and negative with YaRN; the training effect accounts for the gains.

is the source, the update is inference overhead that can be switched off, and the training dynamics deserve the attention.

We measure the two effects separately (Figure 1). The same TTT-trained checkpoint is evaluated twice, as released and with the inner learning rate set to zero at inference. The difference between the two evaluations is the effect of the test-time update. The difference between the update-off evaluation and a matched control is the effect of training with the module. We ran this ablation at a reduced budget of 3.0B tokens with two seeds. The arms, the decision rule and the analysis were fixed before the main runs.

Our claim is that at this budget the gain of In-Place TTT on RULER comes from training with the module. Averaged over seeds, the test-time update changes the 13-task mean by between -0.4 and $+0.2$ points at lengths up to 64k with plain rotary embeddings. At 64k with YaRN it lowers the mean by 2.5 points. Where the TTT arm beats the control, by 1 to 4 points, the checkpoint with the update disabled shows the same advantage. The study also characterises the recipe. The TTT arm trains with a gradient norm two to four orders of magnitude above the control's, almost all of it in the target convolutions. The released inference code applies no clipping, and the clipping threshold reported for the method removes the update rather than bounding it. The released chunk size of 4,096 is the worst of the three sizes we trained.

2 Background

2.1 In-Place Test-Time Training

In-Place TTT (Feng et al., 2026) modifies one layer out of six in the released configuration for Qwen3. In such a layer the MLP down-projection $W \in \mathbb{R}^{d \times d_{\text{ff}}}$ becomes a fast weight. The input is processed in chunks of c tokens, 4,096 in the released configuration. Let h_t be the MLP hidden state at position t and x_t the layer input. The module forms a target per position from a depthwise causal convolution of the layer input, passed through a learned projection P. It then updates the down-projection with an outer-product step,

$$W \leftarrow W + \eta \sum_{t=1}^{c} (P^{\top} \text{conv}(x)_t)\, h_t^{\top}, \tag{1}$$

Table 1: Arms. The update-off rows are the TTT checkpoints evaluated with $\eta = 0$, so no separate model is trained. The 1B-token checkpoints of the two main arms (step 477) are the chunk-4,096 and control references for the chunk ablation.

Arm	Training	Evaluated as	Seeds
Base	none	Qwen3-4B-Base as released	–
Control	3.0B tokens, module disabled	as trained	1, 2
TTT	3.0B tokens, released recipe	as trained ($\eta = 3$)	1, 2
TTT, update off	the TTT checkpoints	$\eta = 0$	1, 2
TTT, η 1 / 6 / clip	the seed-1 TTT checkpoint	$\eta = 1$; $\eta = 6$; $\eta = 3$ with $\tau = 10^{-5}$	1
Chunk 512 / 1,024	1.0B tokens, chunk changed	as trained, and with $\eta = 0$	1

where η is the inner learning rate, 3 in the released configuration. The next chunk is read with the updated W. The update also runs during training, so the shared weights, the convolution and the projection are trained through it. The convolution is zero-initialised and the projection starts at the identity, so a freshly equipped model computes the same function as the base model.

The release has two properties that matter for this study. The inference code applies the update without normalisation or clipping. The paper's appendix, however, describes rescaling any per-chunk delta whose Frobenius norm exceeds a threshold τ, with $\tau = 10^{-5}$ for Qwen3-4B. We added that rule as an option and tested it. The paper's chunk-size ablation on a 1.7B model also prefers chunks of 512 to 1,024 tokens, while the released configuration uses 4,096. The paper does not state the chunk size or inner learning rate behind its Qwen3-4B results.

2.2 Related work

Fast weights written by one network and read by another go back to Ba et al. (2016), and Schlag et al. (2021) showed that linear attention is a fast-weight programmer. Test-time training updates parameters on a self-supervised objective at inference (Sun et al., 2020). For language models it has been applied with nearest-neighbour retrieval (Hardt & Sun, 2023) and as a recurrent layer whose hidden state is a small model (Sun et al., 2024). Titans (Behrouz et al., 2025) learns a memory module with a surprise-based update at test time. In-Place TTT updates existing weights of a pretrained Transformer instead of adding a recurrent block. The same checkpoint can therefore run with or without the update, which is what makes the ablation in this paper possible.

3 What was tested

The design was written down before the main runs and is released with the code. Figure 1 gives the overview and Table 1 lists the arms.

Training. All arms start from Qwen3-4B-Base (Yang et al., 2025; Qwen Team, 2025), a 36-layer model with a native context of 32,768 tokens. The TTT arm uses the released recipe. Fast weights are placed in layers 0, 6, 12, 18, 24 and 30, with a learned target projection, a depthwise Conv1D target, $\eta = 3$ and chunk 4,096. The control trains the same model with the module disabled. The rest is identical. Both arms use the released optimizer settings (AdamW, learning rate 5×10^{-6} with 2% warmup and cosine decay, weight decay 0.1, gradient clipping at 1.0). Both use a global batch of 64 sequences of 32,768 tokens for 1,431 steps, which is 3.0B tokens. The seed changes the data order only. A check at the first step hashes the first micro-batch on all ranks, so same-seed arms consume identical batches. The corpus is public. It packs 3.25B tokens into 32,736-token sequences from PG-19 books, arXiv papers, permissively licensed GitHub code and synthetic retrieval documents (Appendix A).

Evaluation. We use OpenCompass RULER (OpenCompass Contributors, 2023; Hsieh et al., 2024), the framework of the original paper, with its 13 tasks, greedy decoding, 64

generated tokens and base-model prompting. We evaluate 500 instances per task at 4k, 8k, 16k, 32k and 64k. Instances are generated once with a fixed seed, so all arms answer the same prompts and all comparisons are paired. Predictions are re-scored from the saved outputs with RULER's containment rule. Qwen3-4B's native window is 32k. At 64k the primary setting applies YaRN (Peng et al., 2023) with factor 2 to all arms, as the model card recommends. Plain rotary embeddings (Su et al., 2021) at 64k are a secondary setting in which all models are beyond their window. At 4k no update fires, because the prompt is shorter than one chunk, so the 4k comparison isolates the training effect by construction.

The ablation. The update-off variant reproduces the forward pass of the model with the module disabled, with a largest logit difference of 0.0 on held-out text. The active update changes logits by up to 22.7. The attribution at a length is a pair of paired differences. The update effect is TTT minus TTT-update-off. The training effect is TTT-update-off minus control. The two sum to the TTT-minus-control difference. A large update effect would support the stated mechanism. A large training effect would mean that the extra parameters and the training dynamics explain the gain.

Decision rule. Per arm, seed, length and task we report accuracy with a 95% bootstrap interval over instances (Efron, 1979). The headline number is the mean over the 13 tasks, as in the original paper. The rule, fixed before the runs, has two conditions. TTT helps at a length if the paired difference TTT minus control, averaged over seeds per instance, has an interval that excludes zero, and if the difference has the same sign in both seeds. The difference between the two seeds of the same arm is reported as the noise floor.

A corpus artifact and the sensitivity analysis. The synthetic documents end in "Question: …Answer: …" lines after a blank line, which is also the form of RULER's answer prefix. Both trained arms learned to answer two needle tasks, niah_single_2 and niah_multikey_1, by starting a new question block instead of returning the number. The habit is shared by the arms, so paired comparisons stay valid. But the two arms retain it to different degrees at different lengths, and that difference dominates the 13-task means at several lengths. We therefore report all comparisons a second time on the 11 unaffected tasks. This analysis was declared after the seed-1 evaluation and before any seed-2 result; the 13-task analysis stays primary. Appendix B gives the failure-mode counts.

4 Results

4.1 Does In-Place TTT beat the matched control?

Table 2 gives the 13-task means, and Figure 2 plots seed 1. Under the rule, TTT beats the control at 4k, 8k, 16k and 64k with plain RoPE, with seed-averaged margins of 1.1, 1.9, 3.3 and 3.2 points. It does not beat the control at 32k, where seed 1 is 1.8 points behind and seed 2 ties, and the seeds disagree at 64k with YaRN. On the 11 artifact-free tasks the picture is narrower. The two arms are within a point of one another up to 32k in both seeds. TTT leads by 0.6 at 16k and by 3.5 at 64k with plain RoPE, and trails by 2.3 at 64k with YaRN in both seeds (Table 3). The original paper's 32k margin of 0.6 is inside our seed-to-seed spread, which is about a point on the clean tasks at that length.

4.2 Is the gain due to the test-time update?

Figure 3 decomposes the TTT-minus-control difference into the update effect and the training effect, per seed and averaged over seeds, and Table 3 gives the seed-averaged values on the clean tasks. The update effect is near zero at all lengths up to 64k with plain RoPE. Averaged over seeds it lies between -0.4 and $+0.2$ points on the 13 tasks and between -0.3 and $+0.4$ on the clean tasks. The training effect accounts for the gains. It is $+1.1$, $+2.1$, $+3.8$ and $+3.1$ points at 4k, 8k, 16k and 64k plain on the 13 tasks, and $+0.9$ and $+3.1$ at 16k and 64k plain on the clean tasks. At 64k with YaRN the update effect is -2.5 on the 13 tasks and -2.6 on the clean tasks, negative in both seeds. The training effect there is $+0.8$

Table 2: RULER mean over the 13 tasks (500 instances per task; intervals and per-task values in Appendix D). At 4k no update fires, so the TTT and update-off rows coincide.

Seed	Arm	4k	8k	16k	32k	64k plain	64k YaRN
–	Base	93.0	84.1	74.8	75.4	53.7	61.9
1	Control	85.2	82.8	78.7	77.7	47.8	64.6
1	TTT	86.7	84.4	82.7	75.8	50.2	59.5
1	TTT, update off	86.7	84.7	83.1	75.0	51.6	63.8
2	Control	86.5	83.5	78.8	75.5	47.8	62.6
2	TTT	87.3	85.8	81.4	75.5	51.7	64.2
2	TTT, update off	87.3	85.8	81.9	76.0	50.2	64.9

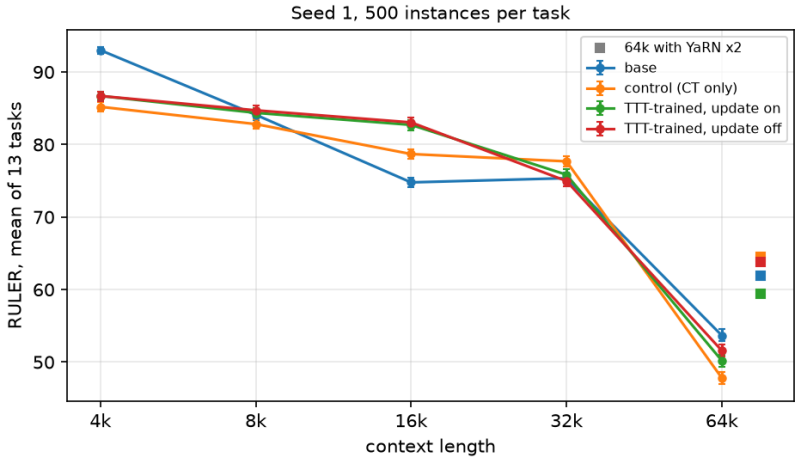


Figure 2: RULER mean over the 13 tasks by context length for seed 1, with 95% bootstrap intervals. Squares at the right are the 64k results with YaRN. The TTT checkpoint with the update disabled (red) tracks the same checkpoint with the update active (green) at all lengths. The gap to the control (orange) is the training effect. Seed 2 is in Appendix G.

and +0.2. This answers the question. The TTT arm beats the control because its weights were trained differently. Switching the update off at inference has no cost at this budget.

The seeds disagree about the update at two settings, and the noise floor locates the disagreement in the update rather than in the weights. At 32k on the clean tasks the update is worth +0.6 in seed 1 and −0.6 in seed 2, with intervals that exclude zero. At 64k with plain RoPE it is worth −1.4 in seed 1 and +1.5 in seed 2. The two seeds' TTT arms differ by 1.2 points at 32k with the update active and by 0.0 with it disabled. At 64k with YaRN they differ by 4.8 points with the update active and by 1.1 without it. The trained weights are reproducible across seeds. The behaviour of the update on top of them is not.

4.3 What does the update do beyond the training length?

At 64k with YaRN the update harms both seeds, for different reasons. In seed 1 it costs 4.4 points. On the frequent-words task the update-off model lists the three frequent words correctly and scores 93.2. The active update falls into a repetition loop ("1. uumoqt - 10 times, 2. uumoqt - 9 times, …") and scores 53.7. The released recipe applies no clipping, and after sixteen accumulated chunk updates generation becomes unstable. In seed 2 the update costs 0.7 points and the frequent-words task is unaffected, so the loop belongs to one trained checkpoint. The loss shared by both seeds is on the hardest multi-key needle task. There the update lowers accuracy from 19.0 to 8.0 in seed 1 and from 19.0 to 4.2 in seed 2, by returning another identifier from the haystack or restating the prompt. With

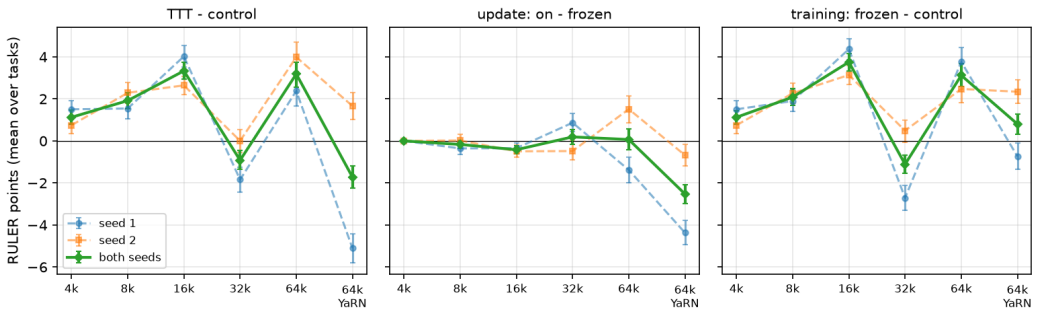


Figure 3: Paired differences on the same RULER instances, mean over the 13 tasks, with 95% bootstrap intervals. Left, TTT minus control. Middle, the update effect (TTT minus TTT-update-off). Right, the training effect (TTT-update-off minus control). Dashed lines are single seeds. The solid line averages the two seeds per instance, the estimate the rule uses. The left panel is the sum of the other two. The same figure for the 11 clean tasks is in Appendix G.

Table 3: Seed-averaged paired differences on the 11 tasks without the format artifact (mean over tasks, 95% bootstrap intervals). "Rule" marks settings where the rule is met, with the sign of the verdict. The noise floor is the difference between the two seeds of the TTT arm with the update active / disabled. The 13-task version is Table 10.

Length	TTT − control	Update effect	Training effect	Rule	Noise floor
4k	+0.2 [-0.0, 0.4]	0.0	+0.2 [-0.0, 0.4]	–	0.1 / 0.1
8k	+0.2 [-0.1, 0.4]	-0.0 [-0.1, 0.1]	+0.2 [-0.1, 0.4]	–	0.1 / 0.1
16k	+0.6 [0.3, 0.8]	-0.3 [-0.5, -0.1]	+0.9 [0.6, 1.1]	yes (+)	0.3 / 0.1
32k	-0.2 [-0.5, 0.3]	-0.0 [-0.4, 0.3]	-0.1 [-0.5, 0.2]	–	1.2 / 0.0
64k plain	+3.5 [2.8, 4.2]	+0.4 [-0.2, 0.9]	+3.1 [2.5, 3.7]	yes (+)	2.3 / 1.4
64k YaRN	-2.3 [-2.8, -1.9]	-2.6 [-3.1, -2.1]	+0.2 [-0.1, 0.6]	yes (−)	2.9 / 0.1

plain RoPE at 64k the update's sign depends on the seed. Its seed-2 gain comes from the simplest needle task, where the update-off model answers most failures with a new question block and the update restores the number (36.2 to 63.4). In both seeds two other needle tasks lose 4 to 8 points under the update (Appendix D).

If the update were the source of the gain, the inner learning rate at inference should matter. Table 4 varies it on the seed-1 checkpoint. At 32k the mean rises by at most 0.9 points at the trained rate and falls at twice that rate. At 64k with plain RoPE any non-zero rate hurts, and the damage grows with the rate. The paper's clipping with $\tau = 10^{-5}$ makes the model indistinguishable from the update-off variant at both lengths. The reason is in the deltas themselves. On held-out 32k text a single chunk changes the down-projection by a Frobenius norm of 0.1 to 0.85. After seven chunks the accumulated change reaches 0.4 to 0.9% of the weight norm in five of the six TTT layers and 2.3 to 2.4% in the last. It grows about linearly with no sign of saturation (Appendix F). The threshold is four to five orders of magnitude below those deltas, so a clip at that value removes the update. Within this recipe no inference-time setting makes the update helpful beyond the training length.

4.4 Do the chunk size and the token budget change the picture?

The chunk size changes how much the active update costs, and the budget changes the sign of the update effect at the training length. Neither changes the conclusion. At 1B tokens the released chunk of 4,096 is the worst of the three sizes with the update active (Table 5). On the clean tasks chunk 512 beats it by 1.2 points at 32k and by 5.1 at 64k with plain RoPE, and chunk 1,024 by 0.3 and 1.1. This is the direction of the original paper's small-model ablation. The update-off rows show where the 32k ordering comes from. With the update

Table 4: Inference-time variants of the seed-1 TTT checkpoint, 13-task mean and paired difference to the update-off variant.

Variant	32k	vs. update off	64k plain	vs. update off
$\eta = 0$ (update off)	75.0	–	51.6	–
$\eta = 1$	75.5	+0.5 [0.1, 0.9]	50.1	-1.5 [-2.0, -1.0]
$\eta = 3$ (as trained)	75.8	+0.9 [0.4, 1.3]	50.2	-1.4 [-2.0, -0.8]
$\eta = 6$	74.1	−0.8	48.2	−3.4
$\eta = 3$, clip $\tau = 10^{-5}$	74.8	-0.2 [-0.5, 0.2]	51.7	+0.2 [-0.2, 0.5]

Table 5: Chunk-size ablation at 1.0B tokens (seed-1 recipe, step 477). 13-task means and paired differences to the 1B control on the 11 clean tasks. "Update off" rows evaluate the same checkpoint with $\eta = 0$.

Model (1B tokens)	32k	vs. control (11)	64k plain	vs. control (11)
Control	77.4	–	45.4	–
Chunk 4,096 (released)	69.5	-2.1 [-2.7, -1.6]	49.1	+3.9 [3.1, 4.7]
Chunk 1,024	76.5	-1.9 [-2.4, -1.3]	50.1	+5.0
Chunk 512	73.4	-1.0 [-1.5, -0.4]	53.5	+9.0
Chunk 4,096, update off	71.3	+0.2 [-0.3, 0.6]	52.7	+7.8 [7.1, 8.5]
Chunk 1,024, update off	77.0	-0.3 [-0.8, 0.2]	53.0	+8.0 [7.2, 8.7]
Chunk 512, update off	73.8	-0.0 [-0.5, 0.5]	53.9	+9.2 [8.4, 9.9]

off, the three chunk sizes are within 0.4 points of one another and of the 1B control on the clean tasks. The active update then costs 2.3 points with chunk 4,096, 1.6 with chunk 1,024 and 1.0 with chunk 512. At one third of the budget the chunk size sets how much the update hurts, and training with the module has not changed the shared weights relative to the control. Beyond the window the same split holds with larger numbers. With the update off, the three chunk sizes are within 1.4 points of one another at 64k and lead the 1B control by 7.8 to 9.2 points on the clean tasks. The active update then costs 3.8 points with chunk 4,096, 2.9 with 1,024 and 0.2 with 512. The 64k advantage of the TTT variants over the 1B control is therefore a training effect, and the smaller chunks look better with the update active only because the update hurts them less.

The budget matters up to 2B tokens and not beyond, within the range we trained. Table 6 evaluates the seed-1 checkpoints at 1B, 2B and 3B tokens. On the clean tasks at 32k the TTT arm trails the control by 2.1 points at 1B and leads by 0.7 at both 2B and 3B. The update effect in this seed moves from −2.3 at 1B to +0.4 at 2B and +0.6 at 3B, while the training effect stays between +0.2 and +0.3. The update becomes harmless at the training length as training proceeds; the shared weights do not pull ahead of the control's on the clean tasks at any budget. Beyond the window the picture is stable from 1B tokens on. With plain RoPE at 64k the TTT arm leads the control by 3.7, 4.0 and 2.4 points at 1B, 2B and 3B, and the update-off checkpoint leads by as much or more at all three budgets. The 64k advantage is a training effect throughout, and the update costs 3.8 points at 1B, 0.2 at 2B and 1.5 at 3B on the clean tasks.

4.5 How does the module change training?

The TTT arm trains in a different regime from the control, which is where its advantage must come from. On identical batches its training loss starts equal to the control's at step 1, because the zero-initialised convolution makes the TTT model equal to the base model. From step 2 on it stays above the control's. The gap shrinks from 0.017 nats over the first 150 steps to 0.007 over the last 143 and does not close within 3B tokens, in both seeds (Table 23). The control's pre-clipping gradient norm stays at 0.2 and is never clipped. The TTT arm's gradient norm starts in the hundreds to thousands and falls to 20 to 30. The released clipping at 1.0 therefore rescales the whole network's gradient at all steps, by a factor of about 50 near the end. On held-out text the final TTT checkpoints have

Table 6: Learning curve of the seed-1 arms at 32k and at 64k with plain RoPE. 13-task means and paired differences on the 11 clean tasks.

Length	Tokens	Control	TTT	TTT, update off	TTT − control (11)	Update effect (11)
32k	1.0B	77.4	69.5	71.3	-2.1 [-2.7, -1.6]	-2.3 [-2.9, -1.8]
32k	2.0B	77.9	75.6	75.1	+0.7 [0.2, 1.3]	+0.4 [0.0, 0.8]
32k	3.0B	77.7	75.8	75.0	+0.7 [0.2, 1.2]	+0.6 [0.1, 1.0]
64k	1.0B	45.4	49.1	52.7	+3.9 [3.1, 4.7]	-3.8 [-4.6, -3.0]
64k	2.0B	47.7	51.7	51.9	+4.3 [3.4, 5.1]	-0.2 [-1.0, 0.5]
64k	3.0B	47.8	50.2	51.6	+2.3 [1.4, 3.1]	-1.5 [-2.2, -0.8]

a gradient norm of 49.6 and 49.2 (seeds 1 and 2). Of this, 49.6 and 49.2 lie in the six target convolutions, which hold 76,800 of the model's 4.0B parameters. The base model with a freshly initialised module already has a convolution gradient 14 times the rest of the network's, so the imbalance exists from the first step. On eight held-out 16k sequences the controls reach 1.823 and 1.822 nats, the TTT arms 1.832 and 1.831, and the base model 1.865. The loss penalty therefore persists outside the training data. At inference the module is cheap. Prefill plus 64 generated tokens at batch size 1 takes at most 2% longer than without the module, and peak memory is 0.38 GB higher (Appendix F).

5 Discussion

The ablation answers the question the published evaluation leaves open. At a 3B-token budget the advantage of an In-Place TTT model over a matched control is present when the fast weights never move. The update is close to neutral at and below the training length and harmful beyond it under the recommended positional scaling. The TTT model is better than the control because it was trained differently.

The training data suggest how. The gradient that reaches the shared weights is rescaled at all steps by a norm that lives in 76,800 convolution parameters. The training loss stays 0.007 nats higher on identical batches. The chunk size does not change the function the update-off model computes, yet it moves RULER by several points. The module therefore acts as a regulariser and an optimiser perturbation during training. Whether a cheaper perturbation, for example the same gradient rescaling without the module, would produce the same training effect is a question this study was not designed to answer.

Our results reinterpret the original paper's numbers rather than contradict them. That paper reports +0.6 at 32k and +4.4 at 64k at roughly ten times our budget and with a 64k training stage we did not run. An effect of 0.6 points at 32k is inside our noise floor. At 64k with YaRN we find the opposite sign and a cause, the unclipped update. The paper describes clipping at $\tau = 10^{-5}$, and on our checkpoints that value switches the update off. Either the paper's models produce deltas five orders of magnitude smaller than ours, or its 64k gains came from models whose update was disabled in effect at inference. The released code and configuration do not let us tell which.

Limitations. Our budget is 3.0B tokens per arm at 32k, against the paper's 20B at 32k followed by 15B at 64k. Our 64k numbers therefore test extrapolation beyond the training length, and effects that need more tokens would be missed. The corpus differs from the paper's, and its synthetic documents introduced the format artifact, which we handle with a sensitivity analysis declared after the seed-1 results. We studied one model, one benchmark with greedy decoding, and two seeds. The learning-rate sweep, the clipping test and the learning curve use seed 1 only, the chunk ablation stops at 1B tokens, and one clipping threshold was tested. Bootstrap intervals cover instance sampling only, and the noise floor shows that single-seed differences of a point or less should be read with care. The upstream code is pinned, with small patches that add configuration validation, metrics logging and the optional clipping. Any difference between the released code and the code behind the paper therefore affects these results as well.

6 Conclusion

An In-Place TTT model differs from its control in two ways, training with the module and adapting at test time. We separated them by evaluating the same checkpoints with the update switched off. At a 3B-token budget on Qwen3-4B, the RULER gains over the matched control come from training with the module. The test-time update contributes at most 0.4 points on average at lengths up to 64k and changes sign between seeds where it is not zero. At 64k with YaRN it costs 2.5 points. This matters for practice because the update can be switched off at inference without loss at this budget. It matters for the method because the training dynamics, rather than the update rule, are where the gain comes from. The recipe trains in a gradient regime set by its target convolutions, pays a persistent loss penalty and uses the worst of three chunk sizes. It applies no clipping at inference, while the threshold reported for it removes the update. A rule that bounds the update without removing it is needed before the update can be trusted beyond the training length. Whether larger budgets or a 64k training stage change the balance is the natural next experiment. The released code runs it from one configuration file.

Reproducibility statement

We release the code, the training and evaluation configurations, and the three patches on the pinned upstream commit. We also release the corpus builder with pinned dataset revisions, the study plan, all per-instance predictions, and the analysis scripts that regenerate the tables and figures of this paper. Paths in the repository are relative, and numbers in the results log name the job and run directory they came from.

References

Jimmy Ba, Geoffrey Hinton, Volodymyr Mnih, Joel Z. Leibo, and Catalin Ionescu. Using fast weights to attend to the recent past. arXiv preprint arXiv:1610.06258, 2016.

Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. Titans: Learning to memorize at test time. arXiv preprint arXiv:2501.00663, 2025.

ByteDance Seed. In-Place-TTT: code release. https://github.com/ByteDance-Seed/In-Place-TTT, 2026.

ccdv. ccdv/arxiv-summarization dataset. https://huggingface.co/datasets/ccdv/arxiv-summarization, 2021.

CodeParrot. github-code-clean dataset. https://huggingface.co/datasets/codeparrot/github-code-clean, 2022.

Arman Cohan, Franck Dernoncourt, Doo Soon Kim, Trung Bui, Seokhwan Kim, Walter Chang, and Nazli Goharian. A discourse-aware attention model for abstractive summarization of long documents. In Proceedings of NAACL-HLT, 2018. arXiv:1804.05685.

Tri Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. arXiv preprint arXiv:2307.08691, 2023.

Bradley Efron. Bootstrap methods: Another look at the jackknife. The Annals of Statistics, 7(1):1–26, 1979.

emozilla. emozilla/pg19 dataset (parquet version of pg-19). https://huggingface.co/datasets/emozilla/pg19, 2023.

Guhao Feng, Shengjie Luo, Kai Hua, Ge Zhang, Di He, Wenhao Huang, and Tianle Cai. In-place test-time training. In International Conference on Learning Representations (ICLR), 2026. arXiv:2604.06169.

Moritz Hardt and Yu Sun. Test-time training on nearest neighbors for large language models. arXiv preprint arXiv:2305.18466, 2023.

Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, Yang Zhang, and Boris Ginsburg. RULER: What's the real context size of your long-context language models? In Conference on Language Modeling (COLM), 2024. arXiv:2404.06654.

Pin-Lun Hsu, Yun Dai, Vignesh Kothapalli, et al. Liger kernel: Efficient triton kernels for LLM training. arXiv preprint arXiv:2410.10989, 2024.

Qianli Ma, Yaowei Zheng, Zhelun Shi, et al. VeOmni: Scaling any modality model training with model-centric distributed recipe zoo. arXiv preprint arXiv:2508.02317, 2025.

OpenCompass Contributors. OpenCompass: A universal evaluation platform for foundation models. https://github.com/open-compass/opencompass, 2023.

Bowen Peng, Jeffrey Quesnelle, Honglu Fan, and Enrico Shippole. YaRN: Efficient context window extension of large language models. arXiv preprint arXiv:2309.00071, 2023.

Qwen Team. Qwen3-4B-Base model card. https://huggingface.co/Qwen/Qwen3-4B-Base, 2025.

Jack W. Rae, Anna Potapenko, Siddhant M. Jayakumar, and Timothy P. Lillicrap. Compressive transformers for long-range sequence modelling. arXiv preprint arXiv:1911.05507, 2019.

Imanol Schlag, Kazuki Irie, and Jürgen Schmidhuber. Linear transformers are secretly fast weight programmers. arXiv preprint arXiv:2102.11174, 2021.

Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding. arXiv preprint arXiv:2104.09864, 2021.

Yu Sun, Xiaolong Wang, Zhuang Liu, John Miller, Alexei A. Efros, and Moritz Hardt. Test-time training with self-supervision for generalization under distribution shifts. In International Conference on Machine Learning (ICML), 2020. arXiv:1909.13231.

Yu Sun, Xinhao Li, Karan Dalal, Jiarui Xu, Arjun Vikram, Genghan Zhang, Yann Dubois, Xinlei Chen, Xiaolong Wang, Sanmi Koyejo, Tatsunori Hashimoto, and Carlos Guestrin. Learning to (learn at test time): RNNs with expressive hidden states. arXiv preprint arXiv:2407.04620, 2024.

An Yang, Anfeng Li, Baosong Yang, et al. Qwen3 technical report. arXiv preprint arXiv:2505.09388, 2025.

Yanli Zhao, Andrew Gu, Rohan Varma, et al. PyTorch FSDP: Experiences on scaling fully sharded data parallel. arXiv preprint arXiv:2304.11277, 2023.

A Training, data and evaluation details

Training. VeOmni (Ma et al., 2025) at the commit pinned by the released In-Place TTT code (ByteDance Seed, 2026), with FSDP2 (Zhao et al., 2023), FlashAttention-2 (Dao, 2023), Liger kernels (Hsu et al., 2024), bf16 compute with fp32 master weights, gradient checkpointing and micro-batch 2. Checkpoints were saved at steps 477, 954 and 1,431 (1B, 2B and 3B tokens). Table 7 lists the configuration.

Table 7: Training configuration (identical for both arms except ttt_mode).

Model	Qwen3-4B-Base (36 layers, hidden 2,560, MLP 9,728)
TTT layers	0, 6, 12, 18, 24, 30
Target	depthwise Conv1D, zero-initialised; learned projection, identity-initialised
Inner learning rate η	3
Chunk	4,096 (512 and 1,024 in the ablation)
Sequence length	32,768
Global batch	64 sequences (2.1M tokens)
Steps	1,431 (3.0B tokens)
Optimizer	AdamW, lr 5×10^{-6}, 2% warmup, cosine decay, weight decay 0.1
Gradient clipping	1.0
Seeds	1, 2 (data order)

Corpus. Table 8 lists the sources. Documents longer than one sequence continue into the next, sequences are shuffled globally with a fixed seed, and a manifest records dataset revisions and counts. All sequences re-tokenise to between 32,704 and 32,737 tokens. The 1,431 steps consume 91,584 of the 99,217 sequences, and 64 sequences are held out for loss checks. The synthetic documents are of two kinds, mixed equally. One is a key-value table of 8-character keys and 6-digit values followed by four "Question: What is the value of key …? Answer: …" lines. The other is a book passage with three to eight planted sentences of the form "The access code for the ⟨object⟩ in ⟨city⟩ is ⟨code⟩." followed by a question about them. The question line after a blank line is what the trained models reproduce on RULER.

Table 8: Training corpus (3.25B tokens in 99,217 sequences of 32,736 tokens).

Source	Dataset	Sequences	Tokens
Books	emozilla/pg19 (Rae et al., 2019; emozilla, 2023)	39,712	1.30B
Papers	ccdv/arxiv-summarization (Cohan et al., 2018; ccdv, 2021)	22,911	0.75B
Code	codeparrot/github-code-clean (CodeParrot, 2022) (permissive licenses, by repository)	27,493	0.90B
Synthetic	key-value and planted-fact retrieval documents	9,165	0.30B

Evaluation. OpenCompass 0.5.4 with the RULER configuration it ships, batch size 1 (the released inference code applies the fast-weight update only to the first sequence of a batch), and the TTT inference classes registered in all workers. Variants (η 0, 1, 6; clipping; YaRN factor 2) are symlinked copies of a checkpoint with an edited configuration file. A calibration run of the base model with 100 instances per task gave a 4k mean of 93.5, against 96.6 reported by the original paper for its continually trained baseline. Task means re-scored from the saved outputs agree with OpenCompass's own.

B The format artifact

Wrong answers were classified into three modes, a new "Question:" block, an empty answer, or other. Table 9 gives the share of new-question answers on the two affected tasks. At 4k both trained arms fail on niah_single_2 in this way about two thirds of the time in seed 1 and half the time in seed 2. At 32k the seed-1 control had almost stopped, while the seed-1 TTT weights continued with or without the update, which is the whole 32k deficit of that seed on the 13 tasks. In seed 2 both arms continued at similar rates and tie at 32k.

Table 9: Share of answers (%) that start a new "Question:" block, by arm and length, on the two tasks affected by the corpus artifact.

Task	Length	Base	Control	TTT	TTT, update off
NIAH single 2	4k, seed 1	0	65	64	64
NIAH single 2	4k, seed 2	0	50	59	59
NIAH single 2	32k, seed 1	0	1	36	41
NIAH single 2	32k, seed 2	0	31	36	36
NIAH single 2	64k YaRN, seed 1	0	3	63	54
NIAH single 2	64k YaRN, seed 2	0	33	39	40
NIAH multikey 1	32k, seed 1	0	41	61	61
NIAH multikey 1	32k, seed 2	0	45	47	49

C Seed-averaged differences on the 13 tasks

Table 10: Seed-averaged paired differences on the 13 tasks (companion to Table 3).

Length	TTT − control	Update effect	Training effect	Rule	Noise floor
4k	+1.1 [0.8, 1.5]	0.0	+1.1 [0.8, 1.4]	yes (+)	0.5 / 0.5
8k	+1.9 [1.5, 2.3]	-0.2 [-0.4, 0.0]	+2.1 [1.7, 2.5]	yes (+)	1.5 / 1.1
16k	+3.3 [2.9, 3.8]	-0.4 [-0.6, -0.2]	+3.8 [3.3, 4.2]	yes (+)	1.3 / 1.1
32k	-0.9 [-1.4, -0.5]	+0.2 [-0.1, 0.5]	-1.1 [-1.5, -0.7]	no	0.3 / 1.0
64k plain	+3.2 [2.6, 3.8]	+0.1 [-0.4, 0.6]	+3.1 [2.5, 3.7]	yes (+)	1.5 / 1.4
64k YaRN	-1.7 [-2.3, -1.2]	-2.5 [-3.0, -2.1]	+0.8 [0.3, 1.3]	no	4.8 / 1.1

Per-seed values on the 13 tasks, TTT minus control at 4k / 8k / 16k / 32k / 64k plain / 64k YaRN, are +1.5 / +1.5 / +4.0 / −1.8 / +2.4 / −5.1 for seed 1 and +0.7 / +2.3 / +2.6 / 0.0 / +4.0 / +1.7 for seed 2. The update effects are 0.0 / −0.4 / −0.3 / +0.9 / −1.4 / −4.4 and 0.0 / 0.0 / −0.5 / −0.5 / +1.5 / −0.7.

D Per-task results

Tables 11 to 16 give per-task accuracies for both seeds at all lengths. The mean row carries the 95% bootstrap interval. NIAH tasks are needle-in-a-haystack variants.

Table 11: Per-task accuracy at 4k.

Task	Base	Control s1	TTT s1	TTT s1 frozen	Control s2	TTT s2	TTT s2 frozen
NIAH single 1	100.0	99.8	99.8	99.8	100.0	99.2	99.2
NIAH single 2	100.0	35.4	36.0	36.0	50.4	40.8	40.8
NIAH single 3	100.0	98.8	98.4	98.4	99.0	99.0	99.0
NIAH multikey 1	93.8	64.6	82.0	82.0	65.6	82.8	82.8
NIAH multikey 2	99.0	98.8	99.4	99.4	99.0	99.2	99.2
NIAH multikey 3	99.6	99.8	99.6	99.6	99.8	99.8	99.8
NIAH multiquery	99.3	98.2	98.7	98.7	98.6	99.2	99.2
NIAH multivalue	99.1	99.8	99.8	99.8	99.7	99.8	99.8
Variable tracking	100.0	100.0	100.0	100.0	100.0	100.0	100.0
Common words	93.9	96.2	96.4	96.4	95.2	96.1	96.1
Frequent words	90.3	89.8	89.1	89.1	90.1	89.9	89.9
QA SQuAD	80.0	76.8	77.2	77.2	77.4	78.6	78.6
QA HotpotQA	54.2	49.6	50.8	50.8	49.8	50.0	50.0
Mean	93.0 [92.5, 93.5]	85.2 [84.6, 85.8]	86.7 [86.1, 87.3]	86.7 [86.1, 87.3]	86.5 [85.8, 87.2]	87.3 [86.6, 87.9]	87.3 [86.6, 87.9]

Table 12: Per-task accuracy at 8k.

Task	Base	Control s1	TTT s1	TTT s1 frozen	Control s2	TTT s2	TTT s2 frozen
NIAH single 1	100.0	99.0	99.6	99.8	100.0	97.8	98.8
NIAH single 2	65.2	63.6	65.0	68.4	72.6	76.2	76.4
NIAH single 3	99.0	97.6	98.8	98.8	97.2	98.6	98.6
NIAH multikey 1	49.6	29.6	44.8	46.2	28.2	54.2	53.2
NIAH multikey 2	96.8	98.2	98.8	99.0	98.6	98.6	98.8
NIAH multikey 3	95.4	95.2	96.4	96.4	95.0	95.8	96.0
NIAH multiquery	97.0	99.0	98.9	98.8	99.1	98.8	98.8
NIAH multivalue	97.1	99.0	98.6	98.7	99.0	98.5	98.4
Variable tracking	99.9	100.0	100.0	99.9	99.9	99.9	99.9
Common words	81.3	87.0	87.0	87.1	88.7	89.1	88.3
Frequent words	95.3	95.9	95.3	95.5	95.7	95.1	95.0
QA SQuAD	66.2	65.8	66.4	65.8	66.4	66.0	66.4
QA HotpotQA	50.4	46.8	47.2	47.0	45.8	47.2	47.0
Mean	84.1 [83.4, 84.8]	82.8 [82.1, 83.5]	84.4 [83.7, 85.1]	84.7 [84.0, 85.4]	83.5 [82.9, 84.2]	85.8 [85.1, 86.5]	85.8 [85.2, 86.5]

Table 13: Per-task accuracy at 16k.

Task	Base	Control s1	TTT s1	TTT s1 frozen	Control s2	TTT s2	TTT s2 frozen
NIAH single 1	100.0	100.0	100.0	100.0	100.0	100.0	100.0
NIAH single 2	20.2	61.4	86.0	89.0	61.8	79.4	79.6
NIAH single 3	94.4	94.2	97.8	98.2	93.6	96.4	97.4
NIAH multikey 1	8.2	8.6	28.2	28.6	9.4	21.8	22.4
NIAH multikey 2	96.2	97.4	98.0	97.6	98.2	98.0	97.4
NIAH multikey 3	85.4	88.8	90.4	92.2	89.2	87.2	91.0
NIAH multiquery	96.1	98.2	98.2	98.0	98.0	97.8	98.0
NIAH multivalue	98.2	99.3	99.1	99.2	99.3	99.2	99.2
Variable tracking	99.9	99.9	100.0	100.0	99.9	99.9	99.9
Common words	65.8	71.4	72.8	72.0	71.3	73.7	72.7
Frequent words	98.3	97.1	97.8	98.0	97.7	97.8	97.9
QA SQuAD	65.0	63.4	63.6	63.2	63.6	64.0	64.8
QA HotpotQA	44.4	43.2	43.4	43.8	42.2	43.6	44.8
Mean	74.8 [74.1, 75.4]	78.7 [78.0, 79.4]	82.7 [82.0, 83.4]	83.1 [82.4, 83.7]	78.8 [78.1, 79.5]	81.4 [80.8, 82.1]	81.9 [81.3, 82.6]

Table 14: Per-task accuracy at 32k.

Task	Base	Control s1	TTT s1	TTT s1 frozen	Control s2	TTT s2	TTT s2 frozen
NIAH single 1	100.0	99.2	100.0	100.0	100.0	99.8	100.0
NIAH single 2	83.2	98.4	63.6	58.6	68.8	63.8	63.6
NIAH single 3	97.6	97.6	98.4	98.2	97.4	97.4	97.4
NIAH multikey 1	45.2	27.0	30.0	29.8	22.0	38.2	38.2
NIAH multikey 2	84.8	88.2	91.6	89.2	91.2	90.0	88.4
NIAH multikey 3	46.0	69.2	75.8	75.0	72.8	65.2	72.6
NIAH multiquery	94.5	94.7	95.9	95.0	94.7	95.3	95.5
NIAH multivalue	96.1	98.0	98.0	98.0	98.2	98.0	98.2
Variable tracking	99.7	99.6	99.7	99.7	99.6	99.5	99.5
Common words	47.1	53.1	52.3	50.1	51.8	52.5	51.6
Frequent words	96.0	95.0	95.3	95.3	95.3	95.8	95.8
QA SQuAD	51.2	51.4	47.8	48.8	50.8	48.0	49.0
QA HotpotQA	38.2	38.4	37.6	37.0	38.8	38.0	37.8
Mean	75.4 [74.6, 76.2]	77.7 [77.0, 78.4]	75.8 [75.1, 76.6]	75.0 [74.2, 75.7]	75.5 [74.8, 76.2]	75.5 [74.7, 76.3]	76.0 [75.2, 76.8]

Table 15: Per-task accuracy at 64k with plain RoPE.

Task	Base	Control s1	TTT s1	TTT s1 frozen	Control s2	TTT s2	TTT s2 frozen
NIAH single 1	86.8	45.8	51.2	46.4	34.2	63.4	36.2
NIAH single 2	83.4	82.8	85.6	85.0	84.4	82.8	83.8
NIAH single 3	95.8	71.2	83.8	88.6	71.4	79.4	83.4
NIAH multikey 1	52.4	50.6	54.0	56.0	50.4	51.6	55.4
NIAH multikey 2	25.2	9.0	18.4	23.6	18.2	15.6	23.4
NIAH multikey 3	2.2	2.6	1.6	3.2	2.2	0.6	2.6
NIAH multiquery	53.5	61.9	64.7	63.6	60.4	64.8	63.4
NIAH multivalue	62.6	73.7	74.8	75.6	74.8	76.0	75.7
Variable tracking	95.3	94.8	96.3	95.2	95.7	96.8	96.0
Common words	11.7	16.9	18.3	16.3	16.0	19.2	16.8
Frequent words	81.3	70.7	59.3	73.5	72.5	81.2	73.7
QA SQuAD	27.6	24.6	25.0	24.6	23.6	22.4	23.8
QA HotpotQA	19.8	17.2	19.6	18.8	17.0	18.8	18.8
Mean	53.7 [52.8, 54.5]	47.8 [47.0, 48.7]	50.2 [49.4, 51.0]	51.6 [50.8, 52.4]	47.8 [46.9, 48.6]	51.7 [50.9, 52.5]	50.2 [49.4, 51.1]

Table 16: Per-task accuracy at 64k with YaRN ×2.

Task	Base	Control s1	TTT s1	TTT s1 frozen	Control s2	TTT s2	TTT s2 frozen
NIAH single 1	99.4	98.4	99.6	98.2	92.6	100.0	98.8
NIAH single 2	70.0	73.0	32.4	38.8	43.6	54.4	52.6
NIAH single 3	96.0	96.4	97.2	97.6	96.2	96.6	97.6
NIAH multikey 1	26.6	11.8	28.2	33.4	15.8	35.8	34.8
NIAH multikey 2	58.6	72.4	76.0	70.0	77.6	72.2	69.2
NIAH multikey 3	12.4	13.2	8.0	19.0	14.8	4.2	19.0
NIAH multiquery	80.6	87.9	90.5	89.0	88.5	89.5	89.1
NIAH multivalue	88.4	94.3	95.4	95.0	94.3	94.9	94.3
Variable tracking	99.0	98.5	98.1	98.9	98.8	98.8	99.1
Common words	13.8	30.9	25.6	27.4	30.5	27.4	27.4
Frequent words	93.9	92.9	53.7	93.2	93.1	93.5	93.1
QA SQuAD	38.0	43.4	43.2	43.8	42.4	42.4	42.8
QA HotpotQA	27.6	26.2	25.0	25.4	25.0	25.2	25.8
Mean	61.9 [61.1, 62.7]	64.6 [63.8, 65.3]	59.5 [58.7, 60.2]	63.8 [63.0, 64.6]	62.6 [61.8, 63.3]	64.2 [63.5, 65.0]	64.9 [64.1, 65.7]

E Sweep, chunk and learning-curve tables

Table 17: Per-task accuracy of the inference-time variants (seed 1) at 32k.

Task	lr 0	lr 1	lr 3	lr 6	lr 3, clip
NIAH single 1	100.0	100.0	100.0	92.2	100.0
NIAH single 2	58.6	61.4	63.6	72.2	57.6
NIAH single 3	98.2	98.0	98.4	98.2	98.4
NIAH multikey 1	29.8	30.6	30.0	31.6	29.6
NIAH multikey 2	89.2	89.6	91.6	92.0	88.6
NIAH multikey 3	75.0	74.8	75.8	67.2	75.2
NIAH multiquery	95.0	95.2	95.9	95.5	95.3
NIAH multivalue	98.0	98.0	98.0	98.3	98.0
Variable tracking	99.7	99.6	99.7	99.6	99.7
Common words	50.1	50.9	52.3	51.1	49.9
Frequent words	95.3	95.6	95.3	79.1	95.5
QA SQuAD	48.8	49.6	47.8	48.4	48.4
QA HotpotQA	37.0	37.8	37.6	38.4	36.6
Mean	75.0 [74.2, 75.8]	75.5 [74.7, 76.3]	75.8 [75.1, 76.6]	74.1 [73.4, 74.9]	74.8 [74.0, 75.6]

Table 18: Per-task accuracy of the inference-time variants (seed 1) at 64k, plain RoPE.

Task	lr 0	lr 1	lr 3	lr 6	lr 3, clip
NIAH single 1	46.4	37.0	51.2	99.4	47.6
NIAH single 2	85.0	85.0	85.6	84.0	85.6
NIAH single 3	88.6	86.8	83.8	79.4	90.4
NIAH multikey 1	56.0	56.0	54.0	52.4	55.4
NIAH multikey 2	23.6	24.0	18.4	20.2	22.8
NIAH multikey 3	3.2	2.4	1.6	0.4	3.4
NIAH multiquery	63.6	64.2	64.7	66.2	63.4
NIAH multivalue	75.6	76.0	74.8	75.4	75.7
Variable tracking	95.2	95.5	96.3	92.3	95.0
Common words	16.3	17.2	18.3	9.1	16.3
Frequent words	73.5	63.7	59.3	2.7	73.3
QA SQuAD	24.6	24.6	25.0	25.4	24.6
QA HotpotQA	18.8	18.6	19.6	19.6	19.0
Mean	51.6 [50.8, 52.4]	50.1 [49.3, 51.0]	50.2 [49.4, 51.0]	48.2 [47.5, 48.9]	51.7 [50.9, 52.6]

Table 19: Per-task accuracy of the 1B-token chunk-size models at 32k.

Task	Control 1B	Chunk 4096	4096 frozen	Chunk 1024	1024 frozen	Chunk 512	512 frozen
NIAH single 1	99.0	98.4	99.8	99.8	100.0	97.6	99.4
NIAH single 2	99.2	29.2	29.8	93.4	84.8	57.2	48.2
NIAH single 3	97.8	96.6	97.0	97.4	97.0	96.6	97.4
NIAH multikey 1	30.0	22.0	18.6	45.2	42.8	31.8	34.8
NIAH multikey 2	88.2	88.8	88.8	87.8	86.6	91.0	88.0
NIAH multikey 3	66.2	51.2	69.0	54.8	67.0	59.8	69.4
NIAH multiquery	94.6	94.8	95.2	95.6	95.5	95.0	94.8
NIAH multivalue	97.8	98.2	98.5	97.9	97.8	97.7	97.8
Variable tracking	99.8	99.8	99.5	99.7	99.6	99.6	99.6
Common words	48.7	44.4	48.2	43.7	47.9	48.2	49.5
Frequent words	94.9	94.4	95.2	93.2	95.3	94.1	95.3
QA SQuAD	50.0	48.2	48.2	48.2	49.2	48.8	47.6
QA HotpotQA	39.6	38.0	39.0	38.0	37.4	37.4	37.6
Mean	77.4 [76.6, 78.1]	69.5 [68.8, 70.4]	71.3 [70.5, 72.0]	76.5 [75.7, 77.3]	77.0 [76.2, 77.8]	73.4 [72.6, 74.2]	73.8 [73.0, 74.6]

Table 20: Per-task accuracy of the 1B-token chunk-size models at 64k, plain RoPE.

Task	Control 1B	Chunk 4096	4096 frozen	Chunk 1024	1024 frozen	Chunk 512	512 frozen
NIAH single 1	23.0	24.6	66.6	93.2	78.2	98.6	89.2
NIAH single 2	82.0	86.6	85.6	84.2	85.2	84.4	86.6
NIAH single 3	62.0	80.6	92.6	81.2	92.6	86.8	90.4
NIAH multikey 1	48.6	48.8	54.2	51.2	55.8	51.6	53.0
NIAH multikey 2	9.8	27.2	19.6	5.0	13.4	9.4	20.6
NIAH multikey 3	2.2	0.2	2.8	0.0	2.6	0.2	2.8
NIAH multiquery	62.5	66.0	64.3	64.0	64.4	63.9	63.0
NIAH multivalue	74.4	75.3	75.8	75.0	76.0	73.2	73.7
Variable tracking	95.0	95.8	95.0	94.2	95.3	95.5	95.6
Common words	16.7	10.1	14.0	5.7	14.0	12.2	14.9
Frequent words	69.5	80.0	72.2	53.9	70.4	77.3	69.1
QA SQuAD	25.6	24.2	24.0	24.4	22.8	22.8	23.4
QA HotpotQA	19.4	19.2	18.4	19.0	18.0	19.0	18.4
Mean	45.4 [44.6, 46.3]	49.1 [48.3, 49.9]	52.7 [51.9, 53.5]	50.1 [49.4, 50.8]	53.0 [52.2, 53.8]	53.5 [52.7, 54.2]	53.9 [53.2, 54.7]

Table 21: Per-task accuracy along the learning curve (seed 1) at 32k.

Task	Control 1B	TTT 1B	TTT 1B frozen	Control 2B	TTT 2B	TTT 2B frozen	Control 3B	TTT 3B	TTT 3B frozen
NIAH single 1	99.0	98.4	99.8	97.6	100.0	100.0	99.2	100.0	100.0
NIAH single 2	99.2	29.2	29.8	99.8	58.4	60.4	98.4	63.6	58.6
NIAH single 3	97.8	96.6	97.0	97.6	98.0	98.0	97.6	98.4	98.2
NIAH multikey 1	30.0	22.0	18.6	30.8	34.0	30.4	27.0	30.0	29.8
NIAH multikey 2	88.2	88.8	88.8	87.6	90.4	88.6	88.2	91.6	89.2
NIAH multikey 3	66.2	51.2	69.0	70.2	73.2	75.4	69.2	75.8	75.0
NIAH multiquery	94.6	94.8	95.2	94.5	95.0	94.7	94.7	95.9	95.0
NIAH multivalue	97.8	98.2	98.5	97.6	98.2	98.0	98.0	98.0	98.0
Variable tracking	99.8	99.8	99.5	99.7	99.7	99.5	99.6	99.7	99.7
Common words	48.7	44.4	48.2	53.0	54.6	49.8	53.1	52.3	50.1
Frequent words	94.9	94.4	95.2	95.1	95.5	95.5	95.0	95.3	95.3
QA SQuAD	50.0	48.2	48.2	51.4	49.0	49.4	51.4	47.8	48.8
QA HotpotQA	39.6	38.0	39.0	38.4	37.0	37.0	38.4	37.6	37.0
Mean	77.4 [76.6, 78.1]	69.5 [68.8, 70.4]	71.3 [70.5, 72.0]	77.9 [77.2, 78.6]	75.6 [74.8, 76.4]	75.1 [74.4, 75.9]	77.7 [77.0, 78.4]	75.8 [75.1, 76.6]	75.0 [74.2, 75.7]

Table 22: Per-task accuracy along the learning curve (seed 1) at 64k, plain RoPE.

Task	Control 1B	TTT 1B	TTT 1B frozen	Control 2B	TTT 2B	TTT 2B frozen	Control 3B	TTT 3B	TTT 3B frozen
NIAH single 1	23.0	24.6	66.6	42.4	71.2	48.0	45.8	51.2	46.4
NIAH single 2	82.0	86.6	85.6	82.8	85.6	84.8	82.8	85.6	85.0
NIAH single 3	62.0	80.6	92.6	71.8	90.0	92.0	71.2	83.8	88.6
NIAH multikey 1	48.6	48.8	54.2	52.6	54.6	55.2	50.6	54.0	56.0
NIAH multikey 2	9.8	27.2	19.6	9.4	15.4	22.4	9.0	18.4	23.6
NIAH multikey 3	2.2	0.2	2.8	1.8	2.0	3.6	2.6	1.6	3.2
NIAH multiquery	62.5	66.0	64.3	60.6	64.5	63.8	61.9	64.7	63.6
NIAH multivalue	74.4	75.3	75.8	72.8	74.8	75.2	73.7	74.8	75.6
Variable tracking	95.0	95.8	95.0	94.5	96.0	95.4	94.8	96.3	95.2
Common words	16.7	10.1	14.0	16.8	18.8	16.1	16.9	18.3	16.3
Frequent words	69.5	80.0	72.2	71.9	53.6	74.1	70.7	59.3	73.5
QA SQuAD	25.6	24.2	24.0	24.6	25.4	23.6	24.6	25.0	24.6
QA HotpotQA	19.4	19.2	18.4	17.6	19.8	19.8	17.2	19.6	18.8
Mean	45.4 [44.6, 46.3]	49.1 [48.3, 49.9]	52.7 [51.9, 53.5]	47.7 [46.8, 48.5]	51.7 [50.9, 52.5]	51.9 [51.0, 52.7]	47.8 [47.0, 48.7]	50.2 [49.4, 51.0]	51.6 [50.7, 52.4]

F Training dynamics and diagnostics

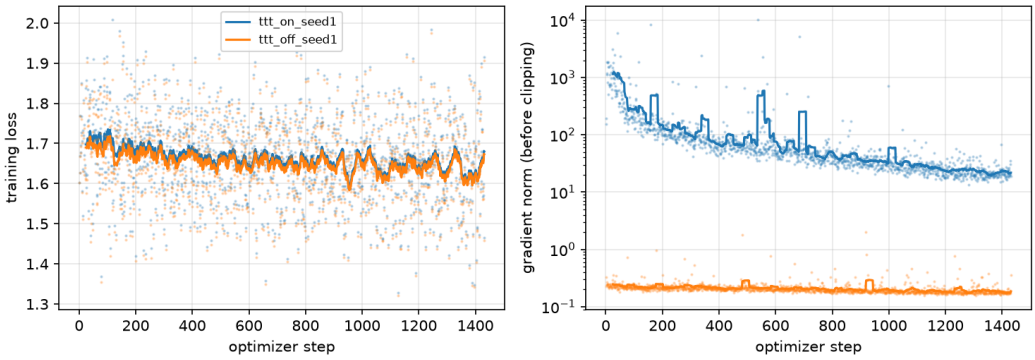


Figure 4: Training loss (left) and pre-clipping gradient norm (right) per optimizer step for the seed-1 arms, which consume identical batches. Points are single steps and lines are running means. Seed 2 is in Appendix G.

Table 23: Same-batch training loss and pre-clipping gradient norm by phase of training (means over steps).

Steps	Tokens	Loss TTT s1 / s2	Loss control s1 / s2	Gap s1 / s2	Grad norm TTT s1 / s2	Grad norm control
1–150	0–0.31B	1.701 / 1.702	1.684 / 1.682	+0.017 / +0.020	546 / 578	0.23
151–477	0.31–1.0B	1.677 / 1.672	1.665 / 1.661	+0.012 / +0.011	138 / 119	0.22
478–954	1.0–2.0B	1.656 / 1.658	1.647 / 1.649	+0.009 / +0.009	93 / 68	0.21
955–1288	2.0–2.7B	1.648 / 1.660	1.641 / 1.653	+0.008 / +0.008	30 / 32	0.20
1289–1431	2.7–3.0B	1.643 / 1.638	1.635 / 1.631	+0.007 / +0.008	22 / 32	0.19

Table 24: Gradient norm by parameter group on two held-out 16k sequences (final checkpoints; the base model with a freshly initialised module as reference, two repeats).

Model	ttt_conv (76.8k)	ttt_proj (39.3M)	down-proj, TTT layers (149M)	other (3.87B)	total
TTT, seed 1	49.58	0.42	0.31	0.95	49.60
TTT, seed 2	49.15	0.42	0.31	0.94	49.16
Control, seed 1	–	–	–	0.99	0.99
Control, seed 2	–	–	–	0.97	0.97
Base + fresh module	17.0 / 18.1	0.00	0.36	1.24	17.1 / 18.2

Table 25: Held-out loss (eight 16k sequences, update active for the TTT arm) and inference cost (prefill of one held-out sequence plus 64 greedy tokens at batch size 1 on one GPU, best of three).

	Seed 1	Seed 2
Held-out loss, base	1.865	1.865
Held-out loss, control	1.823	1.822
Held-out loss, TTT	1.832	1.831
Seconds at 32k, TTT / control	3.86 / 3.78	3.75 / 3.67
Seconds at 64k, TTT / control	7.31 / 7.28	7.24 / 7.09
Peak GB at 32k, TTT / control	15.95 / 15.57	15.95 / 15.57
Peak GB at 64k, TTT / control	23.38 / 23.00	23.38 / 23.00

Table 26: Accumulated fast-weight change $\|W - W_0\|_F$ per TTT layer on eight held-out 32k sequences, after the first and after the seventh update, seeds 1 / 2. $\|W_0\|_F$ is 112 to 126.

Layer	after chunk 1	after chunk 7	relative to W_0 after chunk 7
0	0.13 / 0.10	0.76 / 0.62	0.64% / 0.52%
6	0.26 / 0.22	0.82 / 0.74	0.74% / 0.67%
12	0.10 / 0.11	0.49 / 0.54	0.42% / 0.47%
18	0.16 / 0.16	0.86 / 0.83	0.75% / 0.73%
24	0.21 / 0.18	1.01 / 0.93	0.86% / 0.79%
30	0.85 / 0.72	3.02 / 2.84	2.41% / 2.26%

G Additional figures

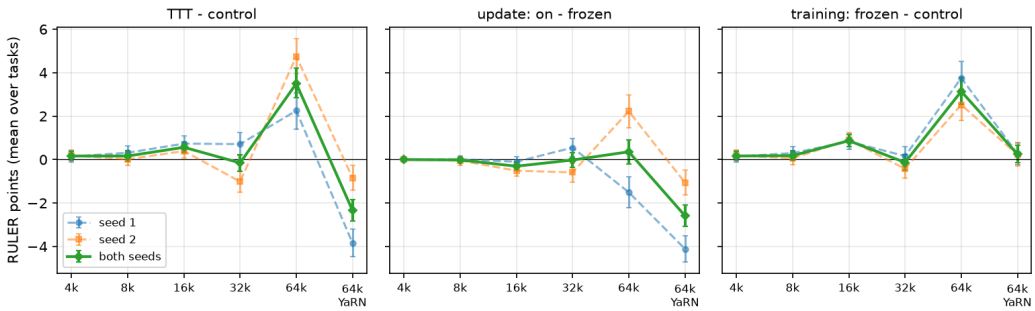


Figure 5: The decomposition of Figure 3 on the 11 clean tasks.

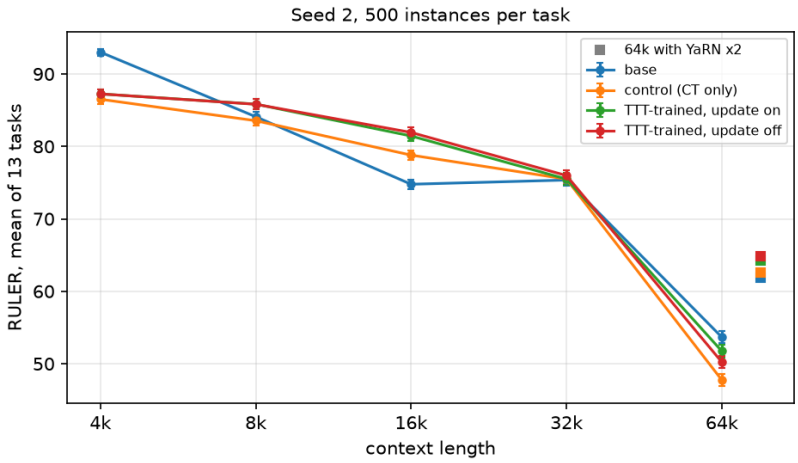


Figure 6: RULER mean by context length for seed 2 (companion to Figure 2).

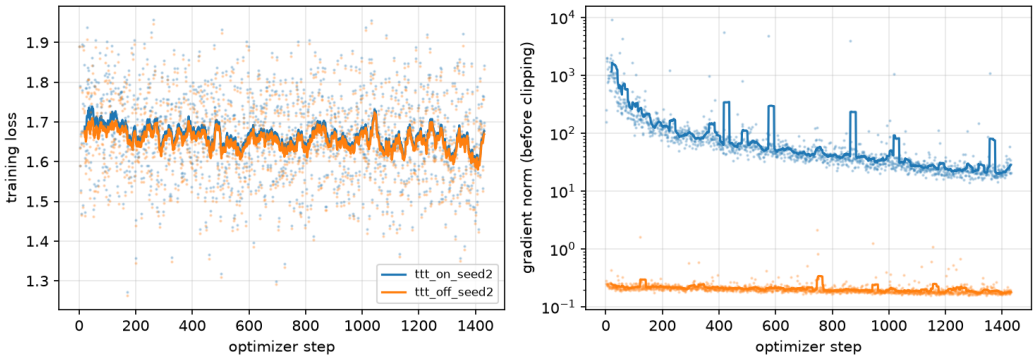


Figure 7: Training loss and pre-clipping gradient norm for the seed-2 arms (companion to Figure 4).